

THE JPG EXPERIMENTS

REGINA FLORES MIR

MAJOR STUDIO II

PROJECT II

FLORR422@NEWSCHOOL.EDU

ALGORITHMS



BIG DATA



GEOMETRY

Objective:

Play with OpenFrameworks and code

EXPERIMENT I:

JPG PARTICLES



GOLAN LEVIN
FLOCCULAR PORTRAITS



EXPERIMENT II:

THE JPG COLLECTION



LEV MANOVICH AND JEREMY DOUGLASS
TIMELINE, 2009



EDWARD TUFTE & ADAM SCHWARTZ
IQ IMAGE QUILTS, CRIME SCENE FINGERPRINTS



CHRISTOPHER BAKER
HELLO WORLD, 2008



LEV MANOVICH
ONE MILLION MANGA PAGES, 2011



The Collection Online / 403,984 records

venus



Sort by Relevance

761 records for
venus

- ☐ Show only artworks on display
- ☐ Show only artworks with images

+ Artist / Maker / Culture

+ Object Type / Material

+ Geographic Location

+ Date / Era

+ Department



Venus and the Lute Player

Titian

(Italian, Pieve di Cadore ca. 1485/90?-1576 Venice)

Date: ca. 1565-70

Medium: Oil on canvas

Accession Number: 36.29

On view in Gallery 607



Venus and Cupid

Lorenzo Lotto

(Italian, Venice ca. 1480-1556 Loreto)

Date:

Medium: Oil on canvas

Accession Number: 1986.138

On view in Gallery 607

CHALLENGE 1:

scrapi, a metropolitan museum collections api

scrAPI.org is an api that grabs object information from the metropolitan museum's [collections](#) website.

Get a random object ([/random](#))

Try `curl scrapi.org/random` in a terminal, or just click on [/random](#)

```
$ curl 'scrapi.org/random'
{
  "CRDID": 12351,
  "accessionNumber": "65.211.3",
  ...
}
```

← Curl is a data transfer protocol. But I need to

Object information ([/object/:id](#))

Try `curl scrapi.org/object/123` in a terminal, or just click on [object/1234](#)

```
$ curl 'scrapi.org/object/123'
{
  "CRDID": 123,
  "accessionNumber": "64.291.2",
  ...
}
```

Searching for object ids ([/search/:terms](#))

You can now search for terms, and get back an array of hrefs to object pages

```
$ curl 'scrapi.org/search/mirror'
[
```

CHALLENGE 2:



```
{
- pagination: {
  next_max_tag_id: "1425402560440628",
  deprecation_warning: "next_max_id and min_id are deprecated for this endpoint; use min_tag_id and max_tag_id instead",
  next_max_id: "1425402560440628",
  next_min_id: "1425435642368844",
  min_tag_id: "1425435642368844",
  next_url: "https://api.instagram.com/v1/tags/metmuseum/media/recent?
  access_token=12119393.ed2506c.d822027f1e7944368df0c4221acd066f&count=100&max_tag_id=1425402560440628"
},
- meta: {
  code: 200
},
- data: [
  - {
    attribution: null,
    - tags: [
      "metmuseum"
    ],
    type: "image",
    location: null,
    - comments: {
      count: 0,
      data: [ ]
    },
    filter: "Normal",
    created_time: "1425435642",
    link: "https://instagram.com/p/xyz9jXgnQT/",
    - likes: {
      count: 3,
      - data: [
        - {
          username: "tzydel",
          profile_picture: "https://instagramimages-a.akamaihd.net/profiles/profile_38902882_75sq_1356566604.jpg",
          id: "38902882",
          full_name: "Tom Zydel"
        },
        - {
          username: "littlestoneprim",
          profile_picture: "https://instagramimages-a.akamaihd.net/profiles/profile_5889722_75sq_1377707680.jpg",
          id: "5889722",
          full_name: "sarah prim-cox"
        }
      ]
    }
  }
]
```



ofxInstagram pagination issue

advanced



ReginaFlores

11h

Hi David! Thank you so much for the reply! Yes I am using your awesome add-on!

The issue I am having is that I define a variable num to be 100. Num is the number of images I would like to get back from Instagram. However `instagram.getImageURL().size()` is always 33. I can't seem to make the URL size to be larger than 33. Do you have any suggestions?

```
instagram.getListOfTaggedObjects("MetMuseum", num);

if (!instagram.getImageURL().empty())
{
    cout<<"this is"<<instagram.getImageURL().size();

    for ( int i = 0; i < instagram.getImageURL().size(); i++)
    {
        getImages.loadFromURL(images[i], instagram.getImageURL()[i]);
    }
}
```

1 Reply ▾



↩ Reply



ofxInsagram pagination issue

advanced



genekogan



david_haylock

9h

+ Reply

the json contains the "next_max_tag_id" which you can pull out with:

```
json["pagination"]["next_max_tag_id"].asString();
```

you can use the result of that as the max_tag_id for the next query, and use that one's next_max_tag_id for the next one, and so on.



Reply



david_haylock

5h

No worries.

@genekogan is spot on, hadn't realised the limit was 33.

@ReginaFlores I'll write something that manages that "id" now should be up in about an hourish



Reply



david_haylock



ReginaFlores

4h

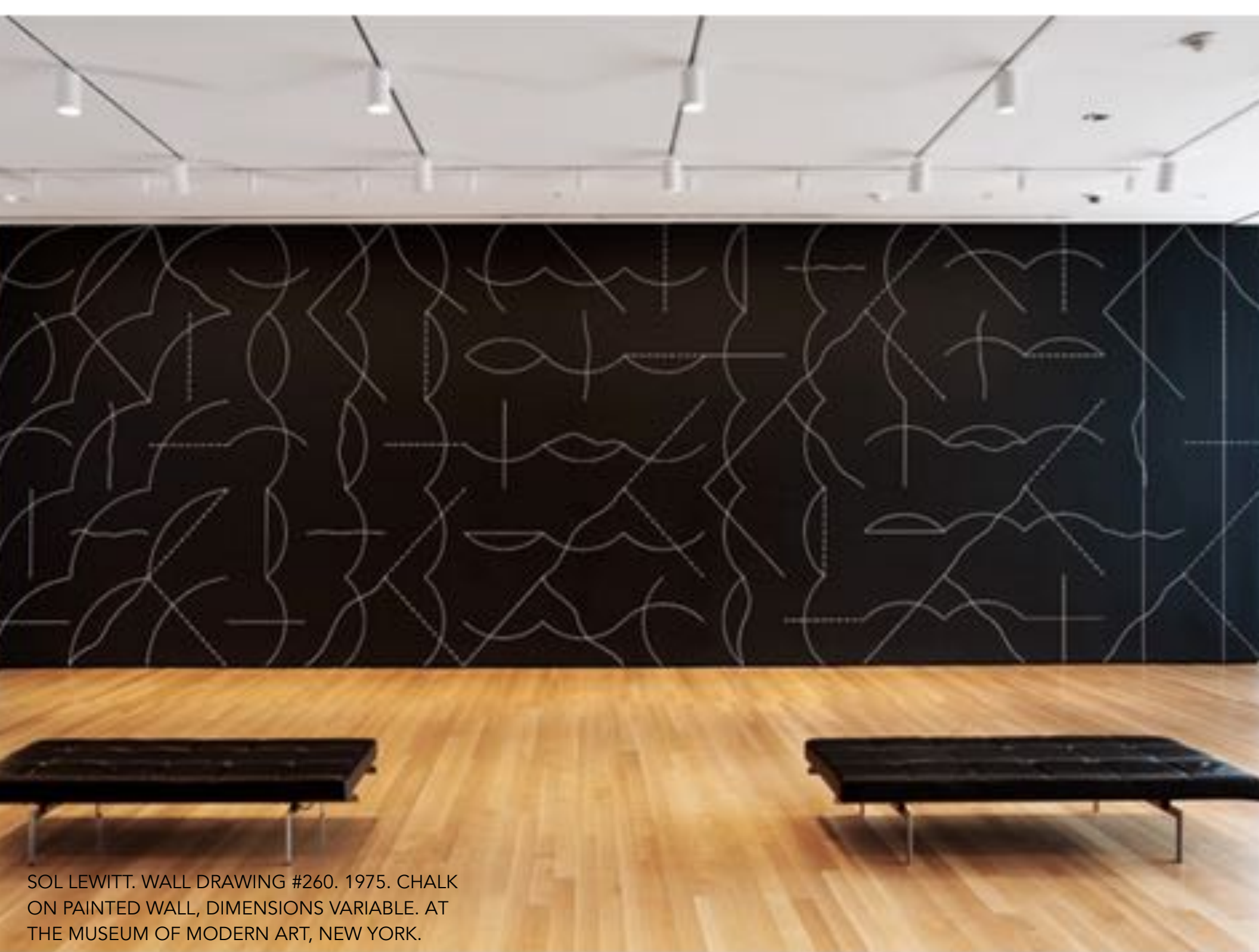
Hi Regina,

There is an example in the repo that lets you cycle through the paginated data. Its a bit basic but it does whats needed. Will try to make a better example at some point today.

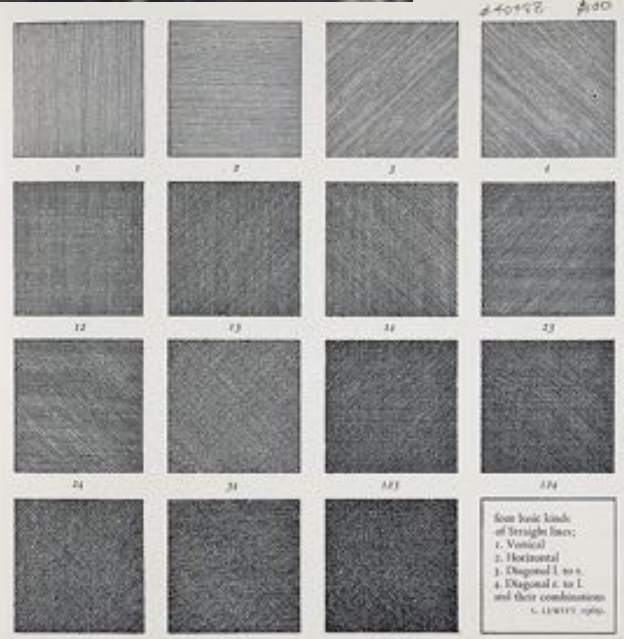
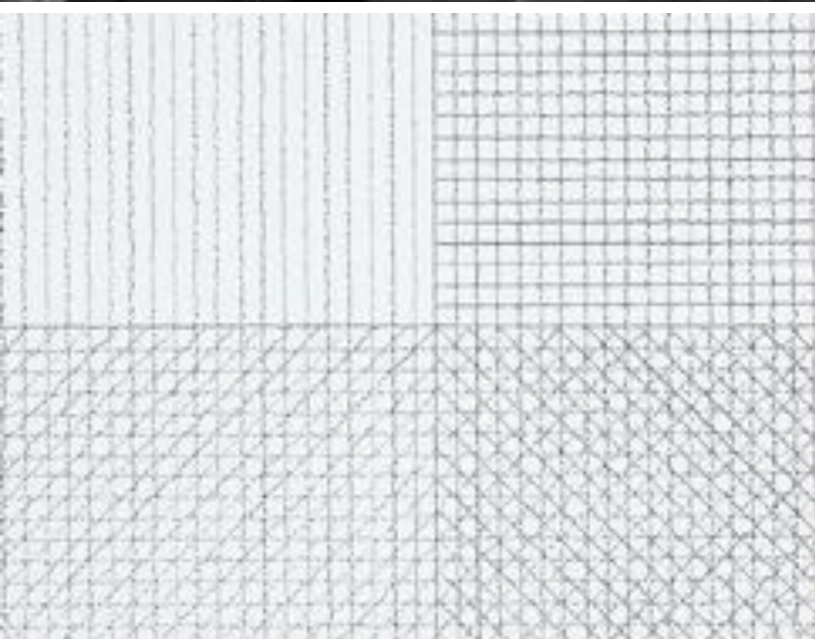


EXPERIMENT III:

JPG ETCHINGS

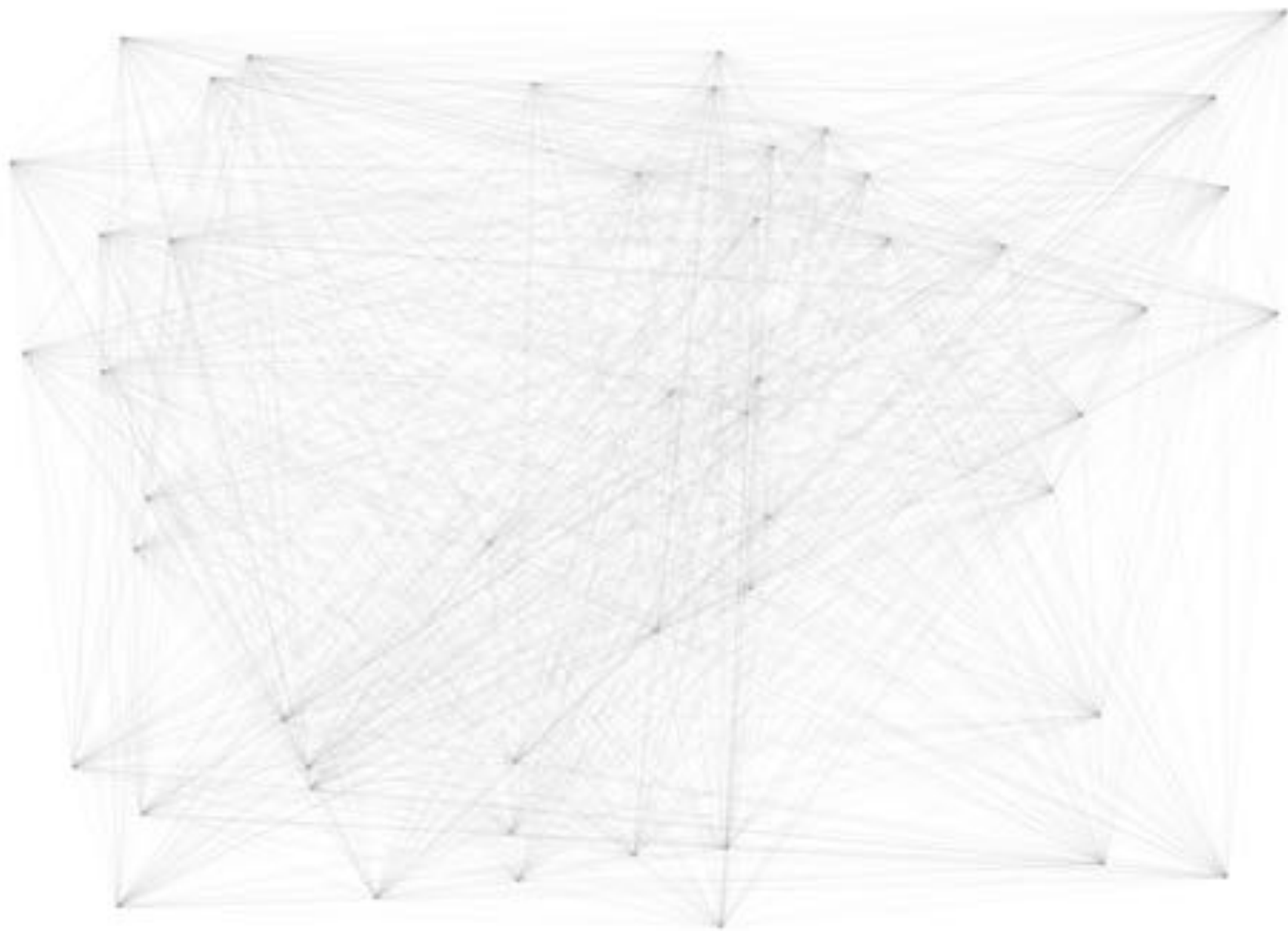


SOL LEWITT. WALL DRAWING #260. 1975. CHALK ON PAINTED WALL, DIMENSIONS VARIABLE. AT THE MUSEUM OF MODERN ART, NEW YORK.

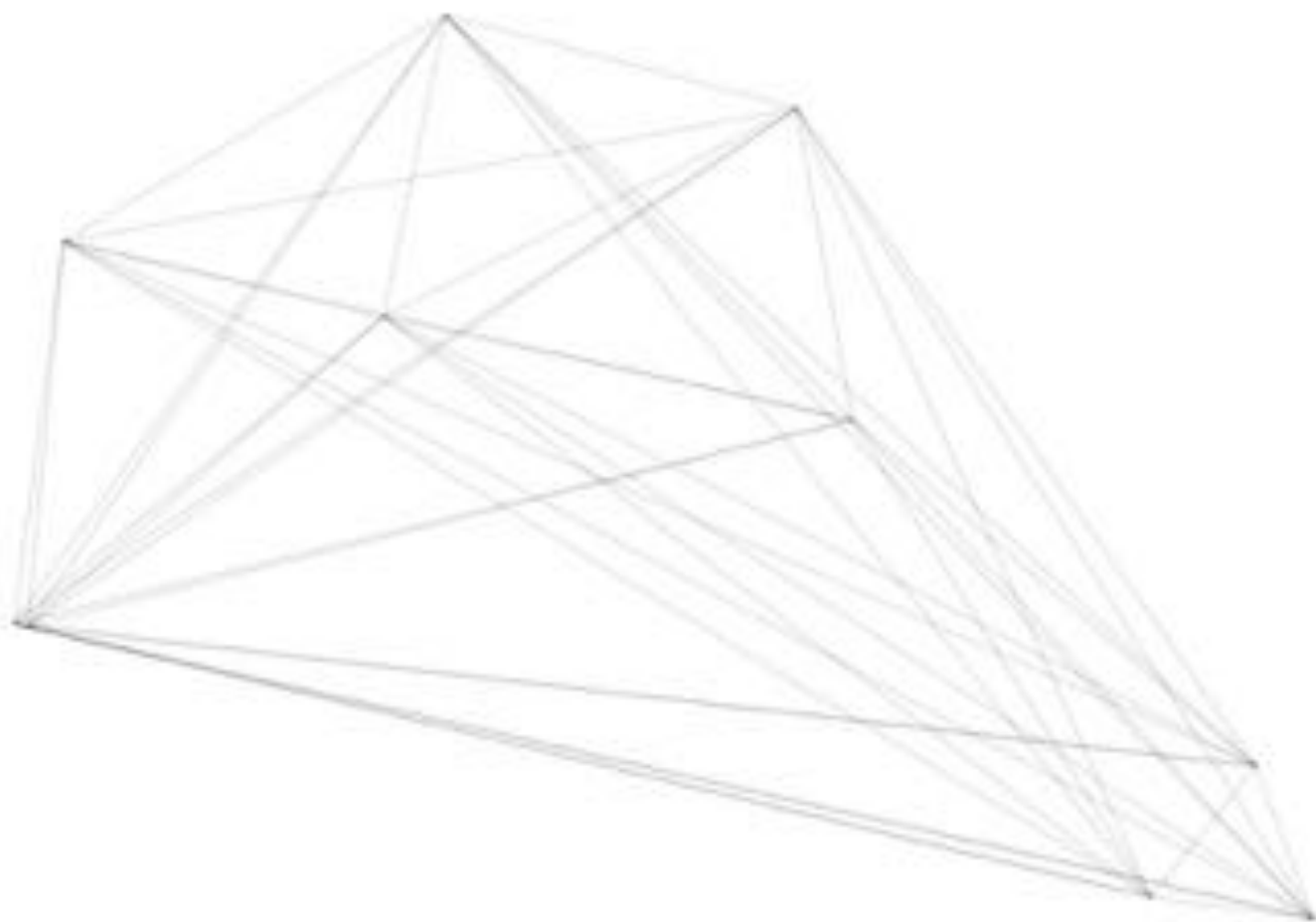




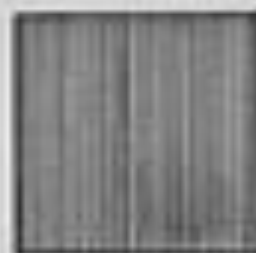
MITCHELL F. CHAN
SOL LEWITT TRANSLATOR
(2013)



$N=50$



$N = 10$



1



2



3



4

1	2	2	1
3	4	4	3
3	4	4	3
1	2	2	1

1

1	3	3	1
4	3	3	4
4	3	3	4
1	2	2	1

2

1	3	3	1
2	4	4	2
2	4	4	2
1	3	3	1

3

1	3	3	1
4	3	3	4
4	3	3	4
1	3	3	1

4

1	4	4	1
2	3	3	2
2	3	3	2
1	4	4	1

5

1	4	4	1
3	2	2	3
3	2	2	3
1	4	4	1

6

2	3	3	2
1	4	4	1
1	4	4	1
2	3	3	2

7

2	3	3	2
4	1	1	4
4	1	1	4
2	3	3	2

8

2	1	1	2
3	4	4	3
3	4	4	3
2	1	1	2

9

2	1	1	2
4	3	3	4
4	3	3	4
2	1	1	2

10

2	4	4	2
1	3	3	1
1	3	3	1
2	4	4	2

11

2	4	4	2
3	1	1	3
3	1	1	3
2	4	4	2

12

3	4	4	3
1	2	2	1
1	2	2	1
3	4	4	3

13

3	4	4	3
2	1	1	2
2	1	1	2
3	4	4	3

14

3	2	2	3
1	4	4	1
1	4	4	1
3	2	2	3

15

3	2	2	3
4	1	1	4
4	1	1	4
3	2	2	3

16

3	1	1	3
2	4	4	2
2	4	4	2
3	1	1	3

17

3	1	1	3
4	2	2	4
4	2	2	4
3	1	1	3

18

4	1	1	4
2	3	3	2
2	3	3	2
4	1	1	4

19

4	1	1	4
3	2	2	3
3	2	2	3
4	1	1	4

20

4	2	2	4
3	1	1	3
3	1	1	3
4	2	2	4

21

4	2	2	4
1	3	3	1
1	3	3	1
4	2	2	4

22

4	3	3	4
1	2	2	1
1	2	2	1
4	3	3	4

23

4	3	3	4
2	1	1	2
2	1	1	2
4	3	3	4

24

THANK YOU

REGINA FLORES MIR
FLORR422@NEWSCHOOL.EDU